

# Architecture Design Document: Workspace Invites & Role-Based Access Control (RBAC)

**Project:** VicharHub

**Author:** Anuj Adhikari

**Status:** Approved for Implementation

**Version:** 1.0

## 1. Executive Summary & Problem Statement

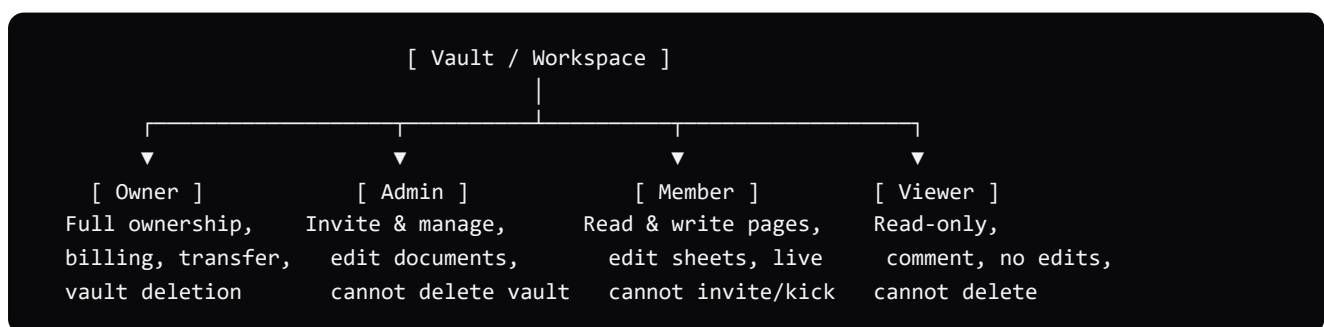
Currently, in VicharHub:

- Vaults are owned by a single `user_id` ( `vaults.user_id = user.id` ).
- Only the creator of a vault can read, modify, or delete its documents and spreadsheets.
- If two teammates or students want to collaborate on the same workspace, they currently have no mechanism to share the vault or control who has administrative power versus editing or viewing rights.

This document outlines the architecture, data models, security guarantees, lifecycle states, and API endpoints for:

1. **Workspace Invites:** Secure invitation flows with time-bounded, cryptographically random tokens and email targeting.
2. **Role-Based Access Control (RBAC):** Fine-grained permission matrices enforcing separation of duties between **Owner**, **Admin**, **Member**, and **Viewer**.

## 2. Core Concepts Explained Simply



The Three Fundamental Entities:

1. **Subject (Who?):** The authenticated user ( `user_id` , `email` , `name` ).
2. **Resource (What?):** The vault, its pages, and collaborative Liveblocks rooms.
3. **Action (How?):** What operation the user wants to perform ( `read` , `create` , `update` , `delete` , `invite` , `manage_roles` ).

## Role Definitions:

- **owner** : The vault creator. Has all capabilities, including destructive actions (deleting the vault) and transferring ownership. There is strictly **1 owner** per vault.
- **admin** : Trusted collaborator. Can invite new members, change roles of members/viewers, remove members, and edit/create pages. Cannot delete the vault or demote/remove the owner.
- **member** : Standard contributor. Can create, edit, duplicate, favorite, and trash pages or edit spreadsheet cells. Cannot invite others, modify vault settings, or manage roles.
- **viewer** : Read-only collaborator. Can view pages, browse the document tree, and observe real-time spreadsheets/docs. Cannot modify titles, content, or settings.

## 3. Permission Matrix (RBAC)

| Action                                                             | Owner | Admin | Member | Viewer |
|--------------------------------------------------------------------|-------|-------|--------|--------|
| <b>View Pages &amp; Spreadsheets</b>                               | Yes   | Yes   | Yes    | Yes    |
| <b>Liveblocks Presence &amp; Cursor</b>                            | Yes   | Yes   | Yes    | Yes    |
| <b>Create New Page / Spreadsheet</b>                               | Yes   | Yes   | Yes    | No     |
| <b>Edit Page Content &amp; Title</b>                               | Yes   | Yes   | Yes    | No     |
| <b>Edit Spreadsheet Cells &amp; Formulas</b>                       | Yes   | Yes   | Yes    | No     |
| <b>Add / Delete Sheets (Tabs)</b>                                  | Yes   | Yes   | Yes    | No     |
| <b>Trash / Restore / Duplicate Page</b>                            | Yes   | Yes   | Yes    | No     |
| <b>Export JSON / CSV Backup</b>                                    | Yes   | Yes   | Yes    | Yes    |
| <b>Invite New Collaborator</b>                                     | Yes   | Yes   | No     | No     |
| <b>Revoke Pending Invites</b>                                      | Yes   | Yes   | No     | No     |
| <b>Change Member Role (Member <math>\rightarrow</math> Viewer)</b> | Yes   | Yes   | No     | No     |
| <b>Promote Member to Admin</b>                                     | Yes   | No    | No     | No     |
| <b>Remove Collaborator (Kick)</b>                                  | Yes   | Yes*  | No     | No     |
| <b>Rename Vault</b>                                                | Yes   | Yes   | No     | No     |
| <b>Delete Vault Entirely</b>                                       | Yes   | No    | No     | No     |
| <b>Transfer Ownership</b>                                          | Yes   | No    | No     | No     |

\*Admins can remove Members and Viewers, but cannot remove the Owner or other Admins.

## 4. Invitation Lifecycle & State Machine

```
stateDiagram-v2
    [*] --> Pending: Admin creates invite with email + role
    Pending --> Accepted: Recipient signs in & clicks accept
    Pending --> Revoked: Admin cancels the invite
    Pending --> Expired: 7 days elapse without acceptance
    Accepted --> [*]: User added to vault_members
    Revoked --> [*]
    Expired --> [*]
```

### Lifecycle States:

#### 1. Creation:

- An Admin/Owner submits recipient's email ( `user@example.com` ) and target role ( `admin` , `member` , `viewer` ).
- A cryptographically secure random token (32 bytes hex) is generated.
- An expiration date is set (default: 7 days from creation).
- Saved with status `pending` .

#### 2. Acceptance:

- Recipient accesses the invite link ( `https://vicharhub.workers.dev/invite/<token>` ).
- If user is not authenticated, they sign in or sign up first.
- Server verifies:
  - Token exists and has status `pending` .
  - `expires_at > current_timestamp` .
  - Optional check: email matches or user accepts with their logged-in account.
- Server inserts recipient into `vault_members` with the assigned role.
- Invite status transitions to `accepted` .

#### 3. Revocation & Expiry:

- If an invite is no longer desired, the Admin can click "Revoke", transitioning status to `revoked` .
- Any access after 7 days is rejected with `410 Gone` (Invite Expired).

## 5. Database Schema (Cloudflare D1 SQLite)

```
-- 1. Vault Members: Junction table associating users with vaults and roles
CREATE TABLE IF NOT EXISTS vault_members (
  id          TEXT PRIMARY KEY,
  vault_id    TEXT NOT NULL REFERENCES vaults(id) ON DELETE CASCADE,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role        TEXT NOT NULL CHECK (role IN ('owner', 'admin', 'member', 'viewer')),
  created_at  INTEGER NOT NULL DEFAULT (unixepoch()),
  UNIQUE(vault_id, user_id)
);

CREATE INDEX IF NOT EXISTS idx_vault_members_user  ON vault_members(user_id);
CREATE INDEX IF NOT EXISTS idx_vault_members_vault ON vault_members(vault_id);

-- 2. Vault Invites: Secure invite tokens and metadata
CREATE TABLE IF NOT EXISTS vault_invites (
  id          TEXT PRIMARY KEY,
  vault_id    TEXT NOT NULL REFERENCES vaults(id) ON DELETE CASCADE,
  inviter_id  TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  email       TEXT NOT NULL COLLATE NOCASE,
  role        TEXT NOT NULL CHECK (role IN ('admin', 'member', 'viewer')),
  token_hash  TEXT NOT NULL UNIQUE,
  status      TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'accepted', 'revoked', 'expired')),
  expires_at  INTEGER NOT NULL,
  created_at  INTEGER NOT NULL DEFAULT (unixepoch())
);

CREATE INDEX IF NOT EXISTS idx_vault_invites_token ON vault_invites(token_hash);
CREATE INDEX IF NOT EXISTS idx_vault_invites_vault ON vault_invites(vault_id);
```

### Migration Backfill:

For all existing vaults in the database:

```
INSERT OR IGNORE INTO vault_members (id, vault_id, user_id, role, created_at)
SELECT 'vm_' || id, id, user_id, 'owner', created_at FROM vaults;
```

This guarantees 100% backward compatibility for all existing user data.

## 6. API Specification

### A. Invites

- `POST /api/vaults/:vaultId/invites`
  - **Required Role:** `owner` OR `admin`
  - **Body:** `{ "email": "teammate@company.com", "role": "member" }`
  - **Response:** `{ "id": "...", "inviteUrl": ".../invite/<token>", "expiresAt": ... }`
- `GET /api/vaults/:vaultId/invites`

- **Required Role:** `owner` or `admin`
- **Response:** List of pending invites for the vault.
- `DELETE /api/vaults/:vaultId/invites/:inviteId`
  - **Required Role:** `owner` or `admin`
  - **Response:** `{ "success": true }` (marks status as `revoked`).
- `GET /api/invites/:token`
  - **Public endpoint:** Returns vault name and inviter name for previewing the invite.
- `POST /api/invites/:token/accept`
  - **Requires:** Authenticated user.
  - **Response:** `{ "vaultId": "...", "role": "..." }` and joins user to vault.

## B. Members & Roles

- `GET /api/vaults/:vaultId/members`
  - **Required Role:** Any member of the vault ( `owner` , `admin` , `member` , `viewer` ).
  - **Response:** `[{ "id": "...", "userId": "...", "name": "...", "email": "...", "role": "admin" }]`
- `PATCH /api/vaults/:vaultId/members/:userId`
  - **Required Role:** `owner` (can change any role) or `admin` (can change `member`  $\rightarrow$  `viewer` ).
  - **Body:** `{ "role": "admin" | "member" | "viewer" }`
- `DELETE /api/vaults/:vaultId/members/:userId`
  - **Required Role:** `owner` or `admin` (or user leaving the vault themselves).

## 7. Security Best Practices

1. **Never Store Plaintext Tokens:** Tokens are hashed with SHA-256 before storing in the database ( `token_hash` ), preventing leaks if the database is dumped.
2. **Server-Side Authorization Everywhere:** The client UI only hides/disables buttons for UX; every single backend endpoint enforces the permission check via SQLite queries ( `getUserVaultRole()` ).
3. **No Self-Demotion of Last Owner:** The vault owner cannot delete themselves or demote their role without transferring ownership first.
4. **Tenant Isolation:** Users can never view or modify pages outside the vaults where they are verified members.